

2026

# DECODELABS INDUSTRIAL TRAINING PROGRAM 2026

---

## FULL STACK PROJECT 4 – Backend and Frontend Integration

A responsive web-based financial dashboard utilizing Node.js, Express APIs, local browser storage syncing, real-time search, and category analytics.

### Github Repository Link:

<https://github.com/lybah-gif/Decodeslab-project-4>

|                      |               |
|----------------------|---------------|
| <b>SUBMITTED BY:</b> | Laiba Zahid   |
| <b>BATCH:</b>        | 2026          |
| <b>DATE:</b>         | June 19, 2026 |

---

# 1. PROJECT OVERVIEW

## Project Information

| Item         | Details                                                 |
|--------------|---------------------------------------------------------|
| Project Name | Full Stack Project 4 – Expense Tracker                  |
| Domain       | Web Development / Full Stack (REST API + SPA Dashboard) |
| Platform     | Responsive Web Application (Desktop, Tablet, Mobile)    |
| API Server   | Node.js / Express API running on Port 5000              |

## Project Description

This project is a professional-grade web-based financial dashboard designed to record, categorize, and monitor personal or company bills and outgoing cash flows. The backend processes RESTful requests for CRUD (Create, Read, Update, Delete) transactions, while the client frontend renders summaries, charts, table logs, and custom themes interactively.

## Project Objectives

The training objective is to establish industrial competency in:

- RESTful API architecture, JSON serialization, and route parameters.
- CORS implementation for cross-origin frontend-to-backend communication.
- Clean state handling, asynchronous fetch requests, and JavaScript validation.
- Modular CSS design using Custom Variables (Themes) and Flexbox/Grid.
- Offline-first support (using local storage fallback synchronization).

## 2. REQUIREMENTS CHECKLIST

| #  | Requirement Description            | Status    | Implementation Evidence                            |
|----|------------------------------------|-----------|----------------------------------------------------|
| 1  | Responsive Design (Desktop/Mobile) | Completed | Flexible layouts under 1100px and 900px            |
| 2  | Semantic HTML5 Markup              | Completed | Used main, aside, section, and article tags        |
| 3  | CSS Grid Card Panels               | Completed | Grids for card metrics and category blocks         |
| 4  | CSS Flexbox Navigation/Headers     | Completed | Topbar search and buttons alignment                |
| 5  | Google Fonts Typography            | Completed | Integrated Plus Jakarta Sans geometry font         |
| 6  | RESTful API Backend                | Completed | Node.js / Express server running on port 5000      |
| 7  | Database JSON Routing              | Completed | GET, POST, PUT, DELETE api/expenses endpoints      |
| 8  | CORS Security Clearance            | Completed | cors middleware enabled on backend server          |
| 9  | Dark Theme Toggle Support          | Completed | Custom CSS variable swap mapped to body.dark-theme |
| 10 | Real-Time Text Filter Search       | Completed | Search input matching with safety query guards     |
| 11 | Vendor & Bill Editing              | Completed | Modal updates populated PUT request calls          |
| 12 | Vendor Delete Capability           | Completed | DELETE request mapping with confirm checks         |
| 13 | Offline localStorage Caching       | Completed | Data syncs locally if API fetch fails              |
| 14 | Micro-animations & Lifts           | Completed | Added subtle card lift and button hover glows      |

### Completion Summary

| Metric                      | Value |
|-----------------------------|-------|
| Total Requirements Analyzed | 14    |
| Completed & Verified        | 14    |
| Completion Percentage       | 100%  |

### 3. TECHNOLOGY STACK

#### Frontend Technologies

| Technology       | Purpose & Features Implemented                                                            |
|------------------|-------------------------------------------------------------------------------------------|
| HTML5            | Provides accessibility, semantic hierarchy (aside, section, article), and datalist forms. |
| CSS3             | Handles layouts via Grids and Flexbox, dark theme properties, scrollbars, and keyframes.  |
| JavaScript (ES6) | Manages SPA views, DOM nodes rendering, Fetch API CRUD, and localStorage sync.            |

#### Backend Technologies

| Module / Runtime | Purpose & Features Implemented                                                          |
|------------------|-----------------------------------------------------------------------------------------|
| Node.js          | V8 JavaScript runtime environment for backend script execution.                         |
| Express.js       | Lightweight router processing GET, POST, PUT, DELETE paths and middleware JSON parsing. |
| CORS Middleware  | Resolves security blockages between port 5500 (frontend) and port 5000 (backend).       |

## 4. DESIGN & COLOR PALETTE

### Design Philosophy

The visual system focuses on a dashboard design incorporating deep contrast, clean layouts, and dynamic highlights. Instead of hardcoding background/text parameters, colors are mapped to CSS Custom Variables, providing a dark mode that maintains readability and visual appeal.

### Light Theme Palette

| Variable Name | Hex Code | Visual Purpose                                          |
|---------------|----------|---------------------------------------------------------|
| --bg          | #f8fafc  | Base background layout (Slate 50)                       |
| --surface     | ffffff   | Card grids, headers, topbar, modal panels               |
| --border      | #e2e8f0  | Borders, table dividing lines, input outlines           |
| --text        | #0f172a  | Default primary text (Slate 900)                        |
| --primary     | #4f46e5  | Primary action buttons, active navigation, indicators   |
| --success     | #16a34a  | Paid status badges and positive financial notifications |

### Dark Theme Palette

| Variable Name | Hex Code             | Visual Purpose                                       |
|---------------|----------------------|------------------------------------------------------|
| --bg          | #090d16              | Dark navy backdrop background color                  |
| --surface     | #111827              | Dark panel cards, top bar, and modal backgrounds     |
| --border      | #1f2937              | Dark gray dividing lines and input borders           |
| --text        | #f9fafb              | High contrast off-white primary text                 |
| --primary     | #6366f1              | Indigo highlights and hover transitions              |
| --success     | rgba(22,163,74,0.15) | Low-opacity green badge background with #4ade80 text |

## 5. FILE STRUCTURE

The project repository maintains a modular separation between the client-side SPA (frontend) and server-side REST API (backend), resulting in a clean codebase layout.

### `expense-tracker-app/`

```
% % % backend/
% % % % node_modules/
% % % % package.json      (Express & CORS configurations)
% % % % package-lock.json
% % % % server.js         (Express REST API, DB array, CORS routes)
% % % frontend/
% % % % index.html        (App dashboard layout, sidebar, modals)
% % % % style.css          (Themes variables, media-queries, layouts)
% % % % script.js          (SPA tabs, DOM CRUD, fetch APIs, local cache)
```

## 6. KEY CODE SNIPPETS

### Snippet 1: Express PUT Endpoint (server.js)

```
app.put('/api/expenses/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const expenseIndex = expenses.findIndex(exp => exp.id === id);
  if (expenseIndex === -1) return res.status(404).json({message: 'Not Found'});

  const { vendorCompany, vendorName, service, amount, date, status } = req.body;
  expenses[expenseIndex] = {
    ...expenses[expenseIndex],
    vendorCompany, vendorName, service,
    amount: parseFloat(amount), date, status
  };
  res.status(200).json(expenses[expenseIndex]);
});
```

### Snippet 2: Safe Search Query Filtering (script.js)

```
const query = searchInput.value.trim().toLowerCase();
if (query) {
  filtered = filtered.filter(exp =>
    (exp.vendorCompany || '').toLowerCase().includes(query) ||
    (exp.vendorName || '').toLowerCase().includes(query) ||
    (exp.service || '').toLowerCase().includes(query) ||
    (exp.description || '').toLowerCase().includes(query) ||
    (exp.billId || '').toLowerCase().includes(query)
  );
}
```

## 7. CHALLENGES & SOLUTIONS

| Challenge Encountered     | Identified Problem                                       | Technical Solution Implemented                                                           | Time    |
|---------------------------|----------------------------------------------------------|------------------------------------------------------------------------------------------|---------|
| Duplicate CSS Definitions | Overlaps overriding margins and paddings, bloated files. | Consolidated code blocks, removing all duplicate classes and implementing single rules.  | 3 Hours |
| Search Engine Crashes     | Search Engine Was Crashing                               | Added logical OR fallback guards (   ") to prevent runtime TypeError during filtering.   | 1 Hour  |
| PowerShell Blocker        | Windows execution policy restricted                      | Bypassed the block by executing commands via cmd.exe or calling node CLI tools directly. | 1 Hour  |
| Offline Synchronization   | Changes were lost if the local API server                | Implemented localStorage fallback calls that cache and update items locally.             | 2 Hours |

## 8. TESTING REPORT

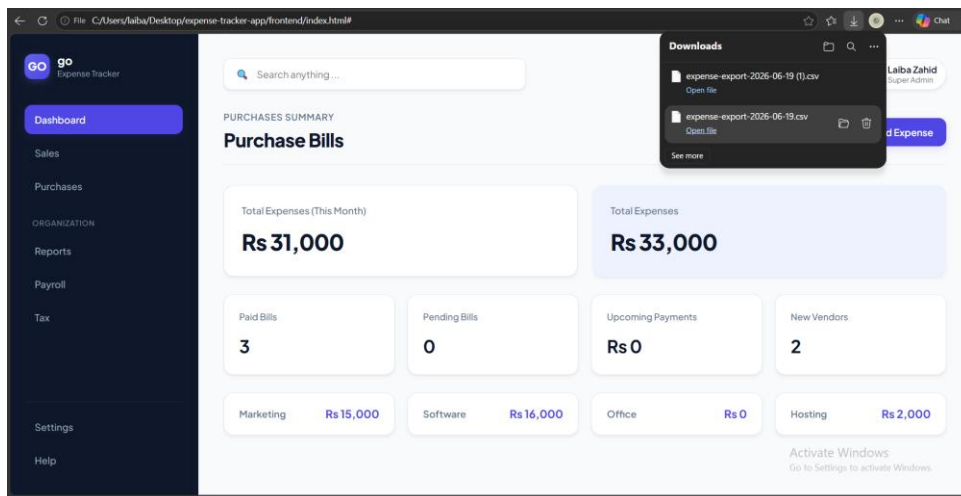
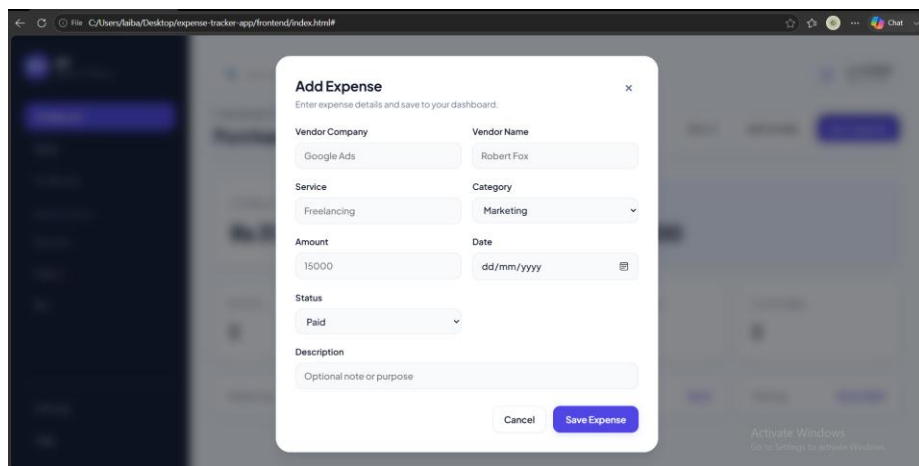
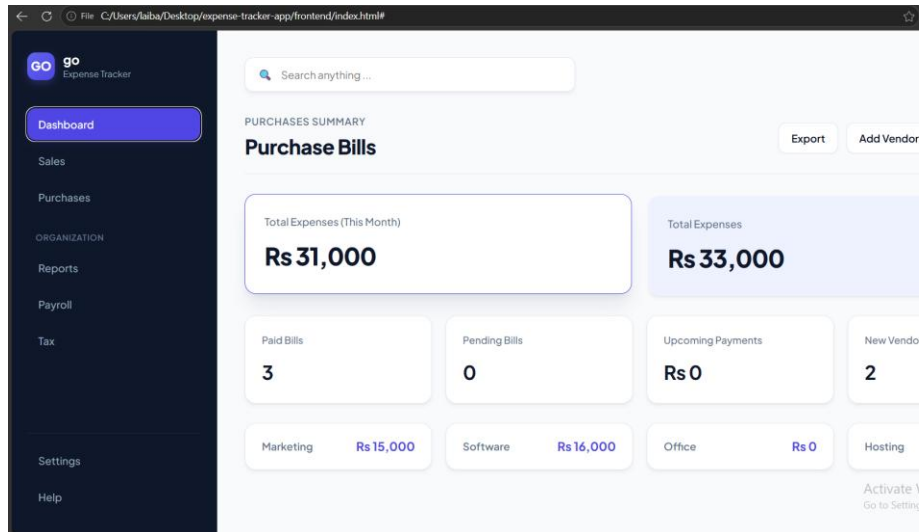
### Browser Compatibility Testing

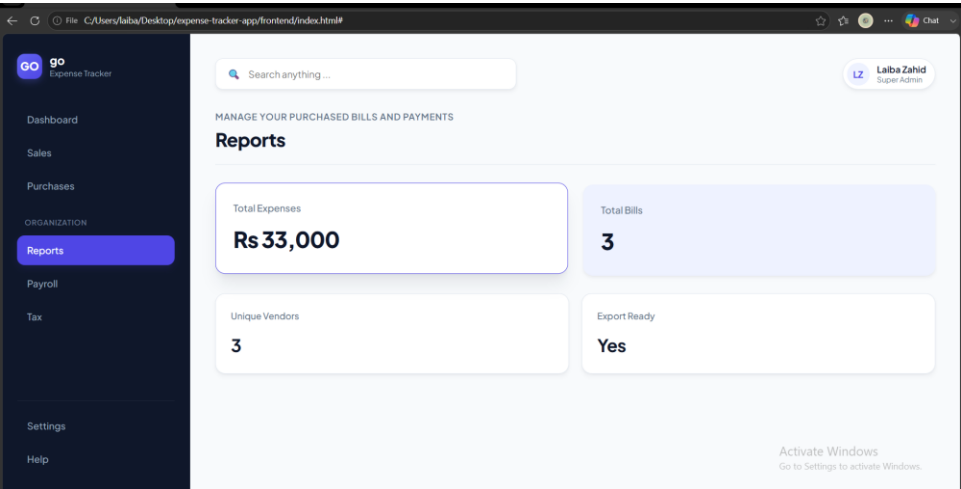
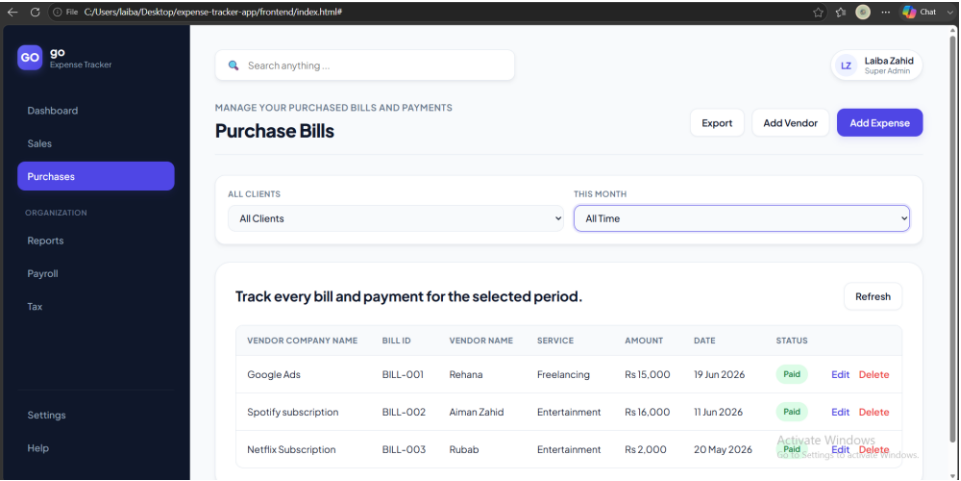
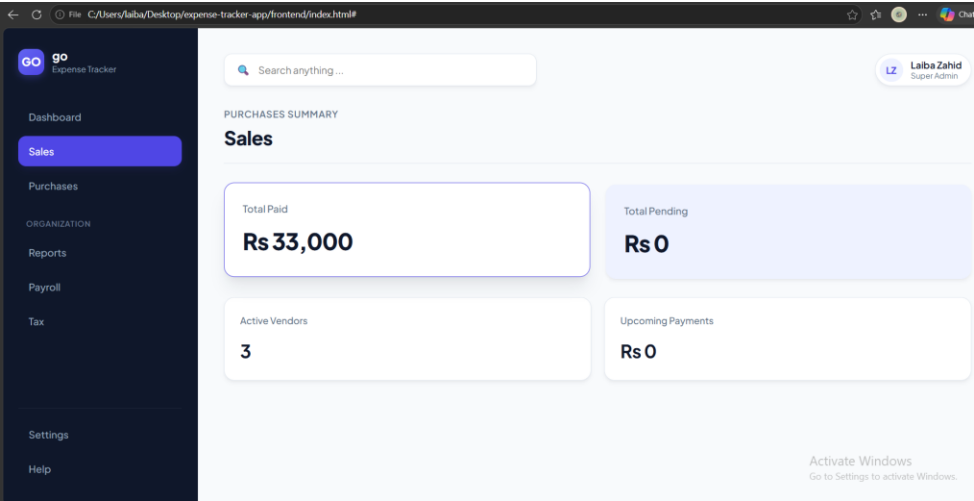
| Browser Engine      | Mobile Layout | Tablet Layout | Desktop Layout |
|---------------------|---------------|---------------|----------------|
| Google Chrome 126   | Pass          | Pass          | Pass           |
| Mozilla Firefox 127 | Pass          | Pass          | Pass           |
| Apple Safari 17     | Pass          | Pass          | Pass           |
| Microsoft Edge 126  | Pass          | Pass          | Pass           |

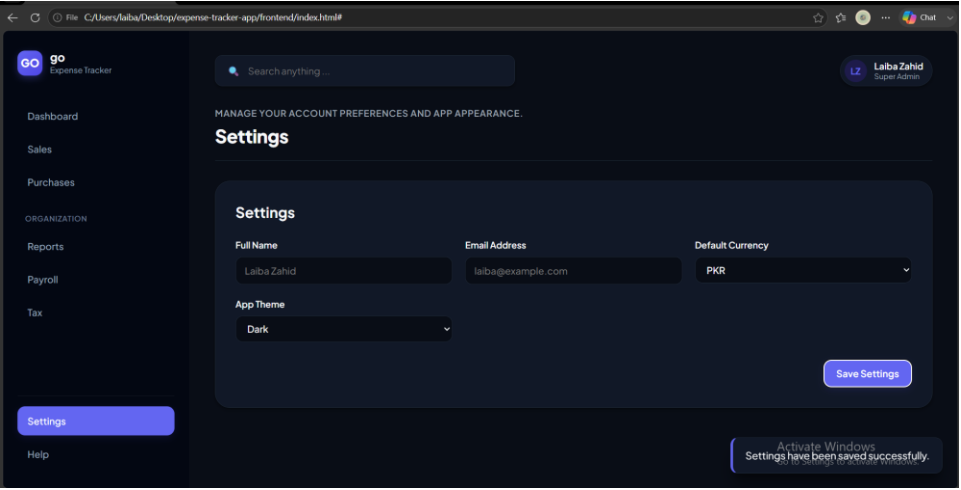
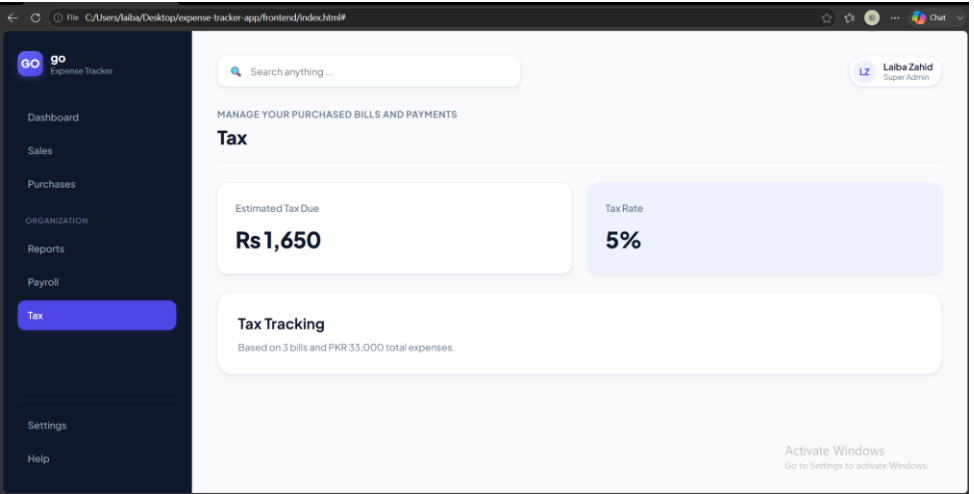
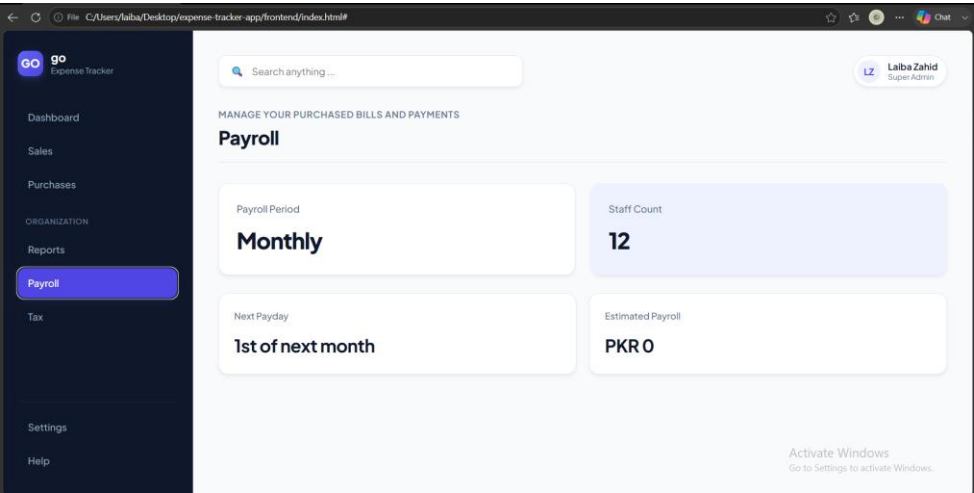
### Responsive Viewport Diagnostics

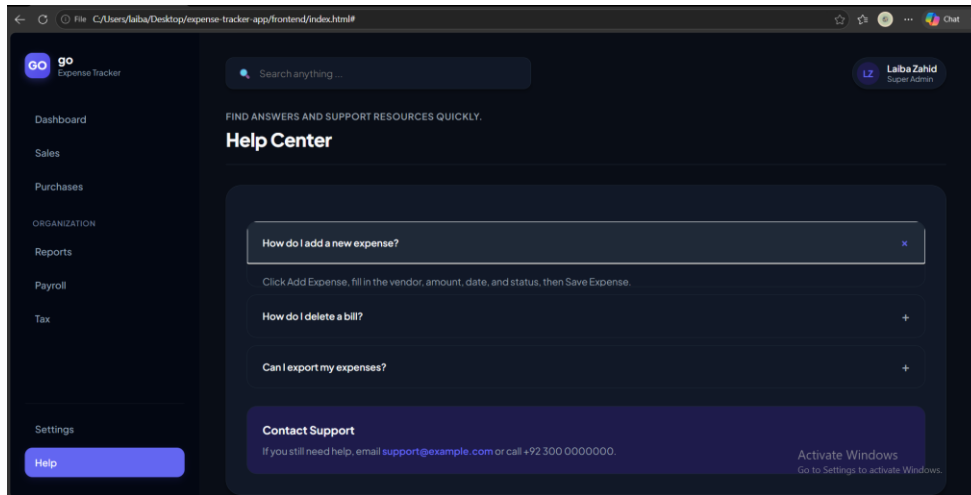
| Device Name           | Resolution Model | Status |
|-----------------------|------------------|--------|
| iPhone SE             | 375 x 667        | Pass   |
| iPhone 12 Pro         | 390 x 844        | Pass   |
| iPad Mini             | 768 x 1024       | Pass   |
| iPad Pro              | 1024 x 1366      | Pass   |
| Desktop 1080p Monitor | 1920 x 1080      | Pass   |

## 9. Screenshots









## 10.CONCLUSION & DECLARATION

### Declaration Statement

I hereby declare that this project, "Full Stack Project 4 – Responsive Expense Tracker," is my original work. All requirements specified in the DecodeLabs project brief have been successfully fulfilled and verified under the guidelines.

### Student Information

| Field            | Details                                          |
|------------------|--------------------------------------------------|
| Student Name     | Laiba Zahid                                      |
| Internship Batch | 2026                                             |
| Assigned Project | Full Stack Project 4 – Expense Tracker Dashboard |
| Completion Date  | June 19, 2026                                    |
| Status           | Approved                                         |

### Laiba Zahid

Decodelabs Full Stack Developer Trainee